

Phase 3 Deep Summary: Git Setup → GitHub → CI/CD

Part 1: Git Fundamentals (From Project Start)

1.1 Initialize the Repository

```
git init
git config user.name "Amrit Keshri"
git config user.email "amritkeshri1805@gmail.com"
```

`git init` creates the local `.git` folder — the entire history database. `git config` (no `--global` flag) sets identity scoped to *this* repo only, stamped into every commit's author field.

1.2 Prevent Committing the Wrong Things — `.gitignore`

```
# Python virtual environment - not portable across OS, reproducible via requirements.txt
venv/
__pycache__/
*.py[cod]
*$py.class

# Generated synthetic data - large, reproducible via generate_synthetic_data.py (fixed RANDOM_SEED=42)
synthetic_data/

# dbt build artifacts - regenerated on every `dbt run`/`dbt build`
target/
dbt_packages/
logs/

# Credentials - defense in depth; dbt's real profiles.yml lives outside the repo in ~/.dbt/
profiles.yml
.env

# IDE / OS noise
.vscode/
.idea/
.DS_Store
Thumbs.db
```

Built *before* the first `git add .`, specifically so the 400MB `synthetic_data/` folder and `venv/` never got staged in the first place.

1.3 Capture the Python Environment

```
pip freeze > requirements.txt
```

Since `venv/` is gitignored, this small manifest is what lets anyone rebuild an equivalent environment (`pip install -r requirements.txt`) without the actual `venv` folder ever being committed.

1.4 The First Commit

```
git add .
git commit -m "Initial project setup: schema docs, synthetic data generator, Snowflake load scripts"
```

1.5 Create the Feature Branch (Before Any dbt Work Began)

```
git checkout -b feature/dbt-staging-layer
```

Created *before* the dbt project even existed, so every subsequent commit — staging layer, testing, marts, everything — landed on this branch, not directly on `master`.

1.6 Commits Made Along the Way (on the feature branch)

```
941e961 Add complete staging layer: 12 models across ecommerce, fintech, and operations domains with custom schema
```

```
0d297f3 Add data tests for stg_customers and stg_orders: unique, not_null, accepted_values, relationships
885c052 Add dbt_expectations range test to stg_reviews
31282df Add marts layer, incremental fct_orders, snapshot on products, custom generic tests, and model contracts
e9eecfd Add dbt Mesh demo (groups, access control), model versioning demo on dim_customer, and additional advanced
b8d8194 Add GitHub Actions CI workflow with environment-aware schema/database routing for CI runs
```

Part 2: Phase 3 — Getting to GitHub and Building CI/CD

2.1 The Core Distinction That Kicked This Off

All of Part 1 happened **entirely locally** — `git commit` never touches the internet. GitHub is a separate, remote hosting service; nothing was backed up or shareable until we explicitly connected and pushed.

2.2 Create the GitHub Repo (done via GitHub's website)

- Created empty (no auto-generated README/.gitignore, to avoid a history conflict)
- Repo: github.com/amritxk/ecommerce-fintech-analytics-platform

2.3 Connect the Local Repo to GitHub

```
git remote add origin https://github.com/amritxk/ecommerce-fintech-analytics-platform.git
```

2.4 Push Both Branches

```
git push -u origin master
git push -u origin feature/dbt-staging-layer
```

`-u` sets up tracking so plain `git push/git pull` know where to go automatically afterward.

Auth troubleshooting hit along the way:

- GitHub password auth is deprecated → needed a **Personal Access Token** instead, used as the password
- First token attempt failed with a `*cached old credential*` issue → fixed via Windows **Credential Manager**, removing the stale `git:https://github.com` entry to force a fresh prompt

2.5 Open the Pull Request

Opened on GitHub's website: base = `master`, compare = `feature/dbt-staging-layer`. Deliberately **not merged yet** — the whole point was to set up CI first and watch it run against this real, open PR.

2.6 Verify the Phase 2 CI Schema Still Existed

```
-- Checked via dbt show --inline querying INFORMATION_SCHEMA.SCHEMATA
-- Confirmed STAGING_DB.SCHEMA_CI_TEMP (created back in Phase 2) was still there and usable
```

2.7 Add GitHub Secrets

Repo → Settings → Secrets and variables → Actions → New repository secret:

Secret	Value
<code>SNOWFLAKE_ACCOUNT</code>	<code>STBPLN0-WK22236</code>
<code>SNOWFLAKE_USER</code>	<code>AMRITKESHRI12345</code>
<code>SNOWFLAKE_PASSWORD</code>	(actual password)
<code>SNOWFLAKE_ROLE</code>	<code>TRANSFORM_ROLE</code>
<code>SNOWFLAKE_WAREHOUSE</code>	<code>TRANSFORM_WH</code>
<code>SNOWFLAKE_DATABASE</code>	<code>STAGING_DB</code>

SNOWFLAKE_SCHEMA	SCHEMA_CI_TEMP
------------------	----------------

2.8 Make the Schema-Routing Macro Environment-Aware

`macros/generate_schema_name.sql` (updated):

```
{% macro generate_schema_name(custom_schema_name, node) -%}

    {%- if target.name == 'ci' -%}
        SCHEMA_CI_TEMP
    {%- elif custom_schema_name is none -%}
        {{ target.schema }}
    {%- else -%}
        {{ custom_schema_name | trim }}
    {%- endif -%}

{%- endmacro %}
```

2.9 Add the Matching Database-Routing Macro (Didn't Exist Before)

`macros/generate_database_name.sql` (new):

```
{% macro generate_database_name(custom_database_name, node) -%}

    {%- if target.name == 'ci' -%}
        STAGING_DB
    {%- elif custom_database_name is none -%}
        {{ target.database }}
    {%- else -%}
        {{ custom_database_name | trim }}
    {%- endif -%}

{%- endmacro %}
```

Together, these two force **every** model — staging or marts — into the single isolated `STAGING_DB.SCHEMA_CI_TEMP` location during CI runs, regardless of each model's normal `+schema:/+database:` config in `dbt_project.yml`.

2.10 The GitHub Actions Workflow

`.github/workflows/dbt_ci.yml`:

```
name: dbt CI

on:
  pull_request:
    branches:
      - master

jobs:
  dbt-build:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: ecommerce_fintech

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dbt
        run: pip install dbt-snowflake

      - name: Create profiles.yml
```

```

run: |
  mkdir -p ~/.dbt
  cat << EOF > ~/.dbt/profiles.yml
ecommerce_fintech:
  target: ci
  outputs:
    ci:
      type: snowflake
      account: "${{ secrets.SNOWFLAKE_ACCOUNT }}"
      user: "${{ secrets.SNOWFLAKE_USER }}"
      password: "${{ secrets.SNOWFLAKE_PASSWORD }}"
      role: "${{ secrets.SNOWFLAKE_ROLE }}"
      warehouse: "${{ secrets.SNOWFLAKE_WAREHOUSE }}"
      database: "${{ secrets.SNOWFLAKE_DATABASE }}"
      schema: "${{ secrets.SNOWFLAKE_SCHEMA }}"
      threads: 4
  EOF

- name: Install dbt packages
  run: dbt deps

- name: Run dbt build
  run: dbt build --target ci

```

2.11 Push the Workflow — Triggering the First Real CI Run

```

git add .
git commit -m "Add GitHub Actions CI workflow with environment-aware schema/database routing for CI runs"
git push origin feature/dbt-staging-layer

```

Second auth issue hit here: push was rejected —

```

refusing to allow a Personal Access Token to create or update workflow
`.github/workflows/dbt_ci.yml` without `workflow` scope

```

Fixed by regenerating the token with **both** `repo` and `workflow` scopes checked.

2.12 CI Ran — Result

■ **dbt CI #1** passed in 48 seconds, triggered automatically by the push (a `pull_request: synchronize` event against the open PR).

Verified, not just trusted, via direct query:

```

-- via dbt show --inline against STAGING_DB.INFORMATION_SCHEMA.TABLES
-- confirmed all 16+ models (staging AND marts, incl. dim_customer_v1/v2, fct_orders,
-- both KPI tables, and the store_failures audit table) landed correctly in SCHEMA_CI_TEMP

```

2.13 Merge the PR

Merged via GitHub's UI as a **regular merge commit** (not squashed — kept the individual commit history intact, since each commit told a real part of the story).

2.14 Bring Local `master` Up to Date

```

git checkout master
git pull

```

This is the exact "pull" scenario described conceptually earlier in the session, now real: local `master` had been sitting untouched at the very first commit this entire time; `pull` brought it up to the merged state that now exists on GitHub.

Still Open (Not Blocking, Just Named Honestly)

- SQLFluff linting — named in original Phase 3 scope, never set up

- A `post-hook` granting `READ_ONLY_ROLE` access — hit an unresolved "does not exist or not authorized" error, set aside
- `store_failures` schema permission — resolved once verified working, but worth double-checking it's still stable